

The Pros and Cons of Polyglot Persistence

The design of a database determines its optimal use. A single database engine is inefficient and insufficient for all data searches. This is where polyglot persistence comes in - it helps to shard data into multiple databases to leverage their power.



Today, we have a large number of databases, which range from document databases like MongoDB and graphs like Neo4j, to search databases like Elasticsearch, caches like Redis and more. All of these databases are great at doing a few things well and other things not so well. For example, Elasticsearch is great for full-text search on large volumes of data, something that cannot be done well in MongoDB.

Polyglot persistence is the way to shard or divide your data into multiple databases and leverage their power together. For example, if you have some data on which a search has to be performed, you can store that data in Elasticsearch because it works on a data structure called *Inverted Index*, which is designed to allow very fast full-text searches and is extremely scalable.

What types of databases can I use?

Document databases (e.g., MongoDB): Document databases are used to store whole JSON documents and query with relevant fields. It's the go-to database for most developers. Document databases are usually bad at doing joins between collections/tables and doing a full-text search.

Graph databases (e.g., Neo4j): Graph databases are used for storing relations between entities, with nodes being

entities and edges being relationships. For example, if you're building a social network and if Person A follows Person B, then Person A and Person B can be nodes and the 'follows' can be the edge between them. Graphs are excellent in doing multi-level joins and are good for features that need the shortest-path algorithm between A and B.

Cache (e.g., Redis): Cache is used when you need superfast access to your data—for example, if you're building an e-commerce application and have product categories that load on every page load. Instead of hitting the database for every read operation (for every page load), which is expensive, you can store it in cache, which is extremely fast for reads/writes. The only drawback of using cache is that it is in-memory and is not persistent.

Search databases (e.g., Elasticsearch): If you want to do a full text search on your data (e.g., products in an e-commerce app), you need a search database like Elasticsearch, which can help you perform the search over huge volumes of data.

Row store (e.g., Cassandra): Cassandra is used for storing time-series data (like analytics) or logs. If you have a use-case that performs a lot of writes, less reads and is non-relational data, then Cassandra is the database to take a look at.